

把“吵闹邻居”揪出来：Kubernetes 混部场景下的干扰检测与 SLO 治理实践

曹阳，JD.com，Senior Engineer



Content 目录

01 背景介绍

02 干扰检测

03 QoS 保障

04 未来规划



Part 01

背景介绍

Kubernetes 在离线混部场景面临的核心问题



背景介绍

- 混部核心挑战：在提升资源利用率的同时，保障在线业务体验与 SLO 。
- 当混部集群出现性能抖动时，最难的是归因，常见影响因素包括：
 - 业务侧变化：流量波动、代码变更、下游依赖抖动
 - 系统侧干扰：节点热点、资源争抢、离线任务抢占
 - 机器 / 微架构级表现：CPI 上升、MPKI 增加、LLC 命中率下降、内存带宽饱和
- 如何快速判定问题是否由混部干扰引起，并及时止损，是混部能否稳定落地的关键问题。



背景介绍



公司 / 品牌	年份	性能变化	业务影响	来源
Amazon	2006	+100ms 页面延迟	-1% sales	
Google	2009	搜索结果页人为增加 100–400ms 延迟	每用户搜索次数下降 0.2%–0.6%	conductor.com
Akamai (行业报告)	2017	网站加载时间增加 100ms	转化率最高下降 7% ；页面慢 2s 时跳出率最高增加 103%	Google Research
Pfizer	2019	站点加载速度提升 38%	跳出率下降 20%	Akamai / PR Newswire
eBay	2020	搜索页加载时间每改善 100ms	“Add to Cart” 数量提升 0.5%	Think with Google PDF
Yelp	2021	P75 FCP 从 3.25s 降到 1.80s ； p75 YPC 从 4.31s 降到 3.21s	转化率最高提升 15%	web.dev
Vodafone	2021	LCP 改善 31%	销售提升 8% ； lead-to-visit 提升 15% ； cart-to-visit 提升 11%	Yelp Engineering
Zalando	2021	页面加载时间改善 100ms	每次会话收入提升 0.7% ，同时跳出更低、互动更高	web.dev
QuintoAndar	2021	优化 Core Web Vitals 并迁移到 Next.js	转化率提升 5% ；每次会话页数提升 87% ；跳出率下降 46%	web.dev
Swappie	2021	平均页面加载时间下降 23% ； LCP 下降 55%	相对移动转化率从 24% 到 34% ；移动收入提升 42%	web.dev
Farfetch	2022	统计分析显示每增加 100ms LCP	平均转化率下降 1.3% ；另一次产品页 A/B 测试把 LCP 降低 600ms+ ，带来 1–5% 转化提升	web.dev
Rakuten 24	2022	A/B 测试中优化版页面更快约 0.4s ， CLS 改善 92.72%	每访客收入提升 53.37% ；转化率提升 33.13% ；客单价提升 15.20% ；退出率下降 35.12%	web.dev
Adobe	2023	LCP 从 7.2s 到 3.4s ； TTI 从 33.3s 到 4.5s ； CLS 从 0.388 到 0	回访提升 14% ；页面停留时长提升 30% ；跳出率下降 12%	web.dev
T-Mobile	2025	持续优化 Core Web Vitals ，重点是 LCP	visit-to-order 提升 60% ；用户站点问题下降 20%	Adobe web.dev



背景介绍

- 常见误区：当应用负责人发现延迟（P95/P99）增加时，通常选择“扩容”更多机器，这只是把问题推开了。因为业务低负载下依然可能存在“资源干扰事件”。真正有效的方法是消除干扰源，而不是无限制增加机器。
- 识别干扰源：Noisy neighbor 导致的性能退化，并不只是 CPU 竞争，更常见的是缓存、内存带宽等共享资源争用。
- 基准测试显示：LLC Miss 升高与业务性能退化具有显著相关性，部分业务的退化幅度接近 3 倍。结合 CPI 指标，可将 LLC Miss 与 CPI 的联合变化作为识别混部干扰的关键观测信号，用于辅助归因 noisy neighbor 问题。

缓存层次越深，访问代价越高

缓存未命中越多，业务越容易落入更高延迟的访存路径。

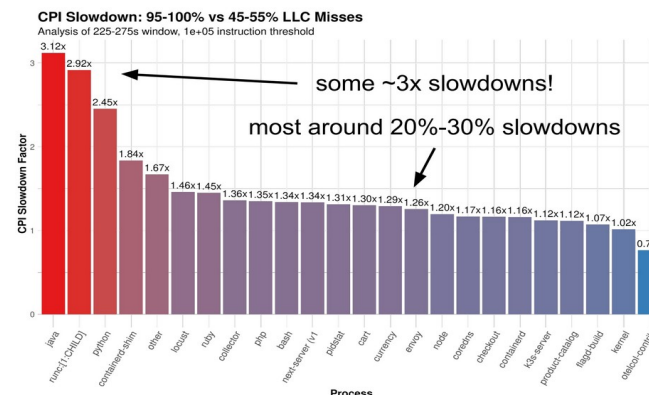
Computers use many levels of caches:

		Access time	Capacity	Managed By
On the CPU	Registers	1 cycle	1 KB	Software/Compiler
	Level 1 Cache	2-4 cycles	32 KB	Hardware
On chip	Level 2 Cache	10 cycles	256 KB	Hardware
	Level 3 Cache	40 cycles	10 MB	Hardware
Other chips and devices	Main Memory	200 cycles	10 GB	Software/OS
	Flash Drive	10-100us	100 GB	Software/OS
	Hard Disk	10ms	1 TB	Software/OS

source : <https://shd.mit.edu/2024/lectures/readings/6191-fall2023/L12.pdf>

高 LLC Miss 会显著抬高 CPI

多数进程慢 20%~30%，少数接近 3x 退化。



source : Noisy Neighbor Collector
<https://github.com/unvariance/collector>

Part 02

干扰检测

数据采集架构、实现算法、案例分析



干扰检测 - 常见干扰类型及典型信号

干扰类型	典型诱因 / 观测信号
CPU 干扰	run queue 堆积、CFS throttling、上下文切换增加、softirq 占用升高、CPU steal（虚拟化场景）
内存 /NUMA 干扰	major page fault 增加、内存回收 /compaction 开销升高、远端 NUMA 访问增加、内存带宽饱和、LLC 争用
磁盘 IO 干扰	I/O latency 升高、队列深度堆积、设备利用率过高、page cache 回收抖动
网络干扰	重传增多、队列拥塞、网络 softirq 占用升高、conntrack 压力增大
调度 / 隔离失效	CPU 绑核失效、cpuset/cgroup 隔离不足、优先级配置不当



干扰检测 - 关键指标采集

识别混部干扰，需要同时观察业务、系统与微架构信号。

业务结果层

- TP99/TP999 时延：越低越好
- Availability / 错误率：可用性越高、错误率越低越好

资源压力层

- 节点 / Pod CPU、内存、网络、I/O 使用率
- 在线 / 离线 CPU 占用
- 内存带宽
- ...

干扰信号层

- PSI (cpu / mem / io)
- CPI
- MPKI / LLC Miss
- ...

关键干扰信号解释

业务指标反映业务结果，资源指标反映压力，PSI / CPI / MPKI 用于确认是否存在资源争用。

PSI

直接衡量任务因 CPU / 内存 / I/O 争用而 stall 的时间占比。some 表示至少部分任务 stall，可作为早期争用信号；full 表示所有非 idle 任务同时 stall，说明瓶颈已较严重。

CPI

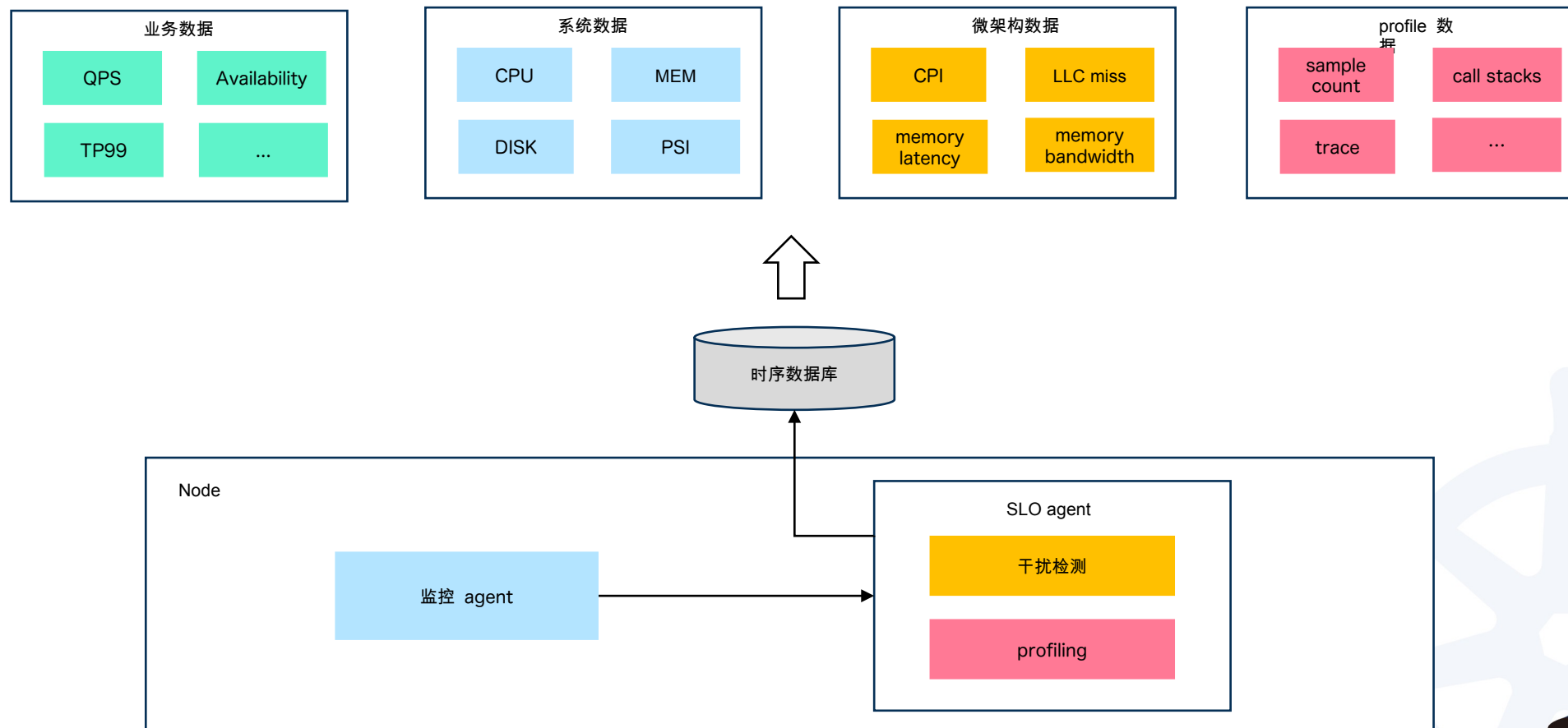
平均每条指令消耗的 CPU 周期数，反映执行效率。持续升高通常意味着受到锁竞争、访存开销、NUMA 访问或 cache miss 等影响。

MPKI / LLC Miss

每千条指令的缓存未命中次数，反映缓存局部性与访存压力。持续升高通常表示缓存争用增强、访存变慢，并常与 CPI 上升同时出现。

关键判定：只有在在线业务指标异常与同一时间窗口出现某类资源压力（CPI/PSI/内存带宽等）同时满足才判定为混部干扰。这样能有效区分是业务自身波动 / 代码问题（只有业务指标异常，没有资源压力特征）还是混部资源争抢问题。

干扰分析 - 数据采集架构



干扰检测 - 实现算法

健康分数定义

将“业务性能退化”和“系统受压信号”统一映射为一个 0~1 的健康分数。分数越接近 1 表示系统越健康，越接近 0 表示干扰或退化越明显。

指标定义

指标集合为 M 。对每个指标 i ，给定观测值 x_i 、阈值区间 $[L_i, U_i]$ 和权重 w_i 。

单指标归一化

将不同指标统一映射为 $[0,1]$ 区间内的惩罚项。

成本型（越大越差）：如 `tp99/cpu/mem/throttle/psi`

$$p_i = \text{clamp}((x_i - L_i) / (U_i - L_i), 0, 1)$$

效益型（越大越好）：如 `availability`

$$p_i = 1 - \text{clamp}((x_i - L_i) / (U_i - L_i), 0, 1)$$

指标聚合

对有效指标集合 M' 做权重归一化，得到原始分数 Score_raw 。

$$\hat{w}_i = w_i / \sum_{j \in M'} (w_j)$$

$$\text{Score_raw} = \text{clamp}(1 - \sum_{i \in M'} (\hat{w}_i \cdot p_i), 0, 1)$$

平滑处理

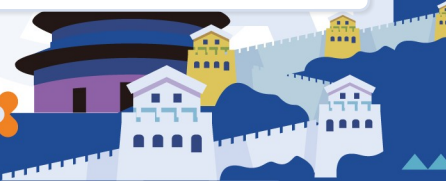
为降低瞬时抖动对结果的影响，采用两阶段平滑：`median-of-3` 消除短时毛刺，`asymmetric EWMA` 对退化更敏感、对恢复更平滑。

Output $\rightarrow$ Health Score $\in [0,1]$



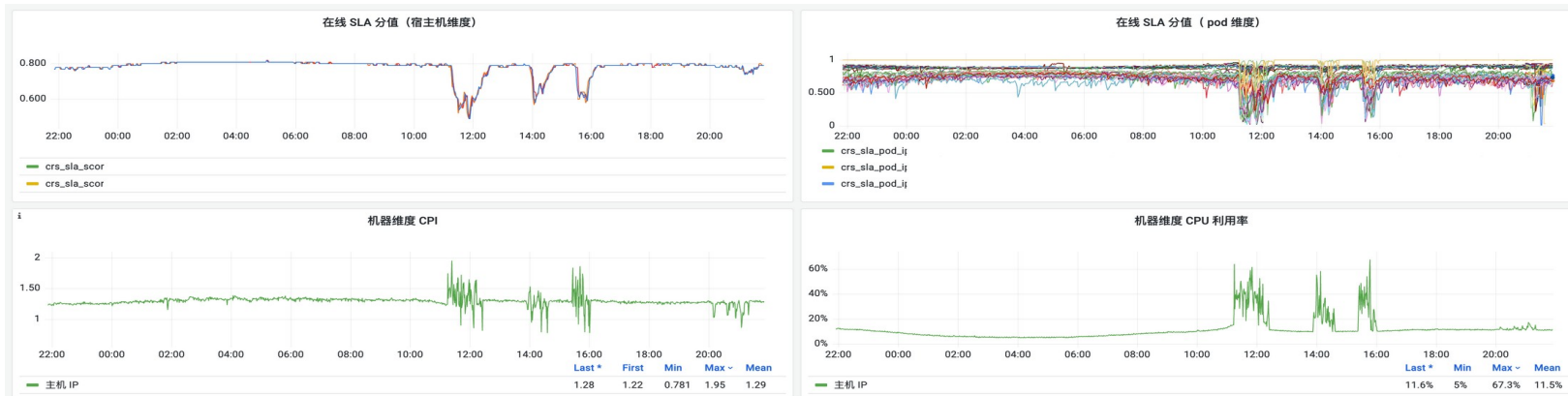
干扰检测 - 最佳实践

建立“采集 → 检测 → 归因 → 处置 → 验证”的治理流程，用业务、系统与微架构信号快速识别并止损混部干扰。



干扰检测 - 案例分析

基于健康度分数的干扰感知：



基于健康度分数，联动离线任务实时压制：



LLM

KCD
GROWING CLOUD NATIVE TOGETHER
BEIJING

集群视角：



应用视角：



Part 03

QoS 保障

QoS 保障机制、体系化建设、动态内存带宽调控



QoS 保障 - 概述

QoS 保障定义

混部集群 QoS 保障：当在线业务与离线作业共享同一批机器时，通过监控、识别、隔离与调度等机制，优先保障在线业务的延迟、错误率、可用性等关键指标不受资源争抢影响，同时尽可能将闲置资源给离线使用。

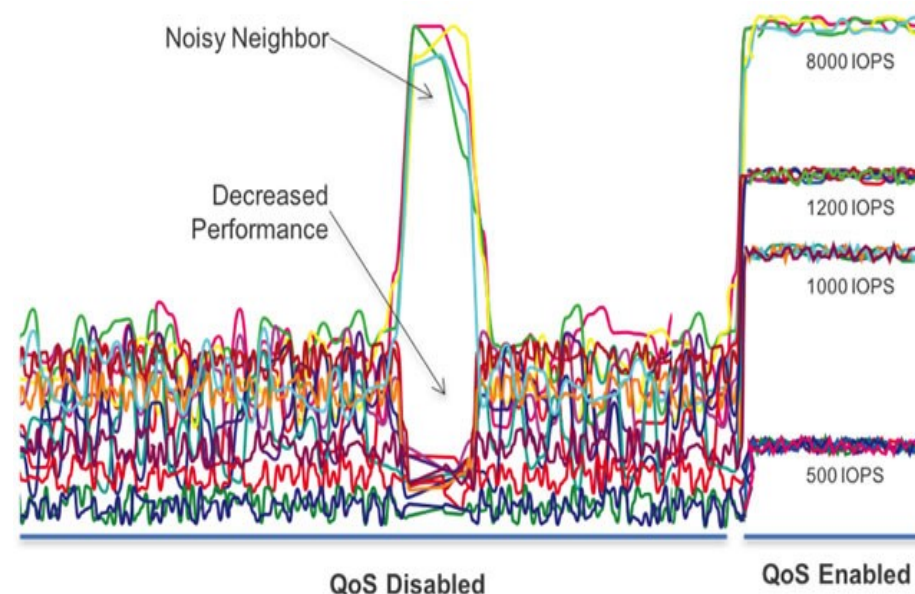
保障对象

- 在线业务：时延 / SLO
- 离线作业：吞吐 / 时效

关键手段

- 干扰检测
- 资源限制 / 隔离
- 调度感知与优先级

核心挑战：在不破坏在线 SLO 的前提下，抑制 noisy neighbor 带来的性能抖动，实现更高的资源利用率。



QoS 保障 - 体系化建设

QoS 的核心是差异化保障，结合可观测与干扰感知、调度与隔离能力，对关键在线任务优先兜底，对可让渡离线负载弹性回收，实现“体验不降、利用率提升”。



QoS 保障 – 支持差异化保障的资源类型与机制

不同资源争用类型，需要匹配不同的 QoS 控制手段。

资源类型	典型竞争	保障机制
CPU	核心 / 调度竞争	SMT 隔离；离线调度类（类 CFS）；CPU 配额与优先级
LLC	缓存争用	Intel RDT CAT；AMD QoS L3 CAT
内存	容量 / 回收 / OOM	内存静态分配与预留（如 Kata）；OOM Kill 优先级
磁盘 I/O	吞吐 / 时延抖动	IOPS / BPS 限速；I/O QoS
网络	带宽 / 排队竞争	全网流量控制（高 / 中 / 低优先级）；单机带宽流控与优先级
内存带宽	访存带宽争用	Intel RDT MBA；AMD QoS MBA
NUMA	远端访问 / 绑核失配	NUMA 亲和；绑核与调度策略



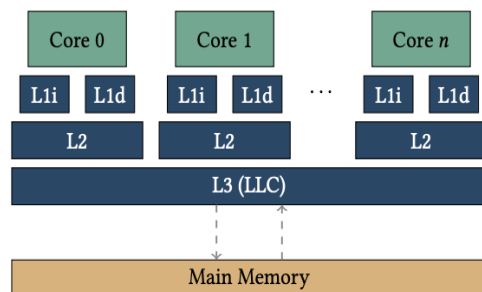
QoS 保障 - 动态内存带宽调控

内存带宽竞争问题

访存密集型离线作业与时延敏感在线服务混部时，内存带宽一旦接近饱和，在线延迟会快速上升。

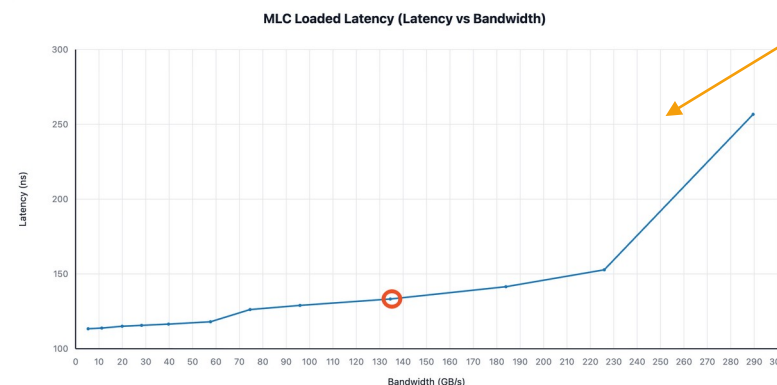
共享路径

在线与离线任务共享 LLC 与主存带宽，访存路径上的竞争会直接放大访问时延。



带宽拐点

内存带宽逼近上限后，访问延迟会从相对平稳区快速进入陡升区。



进入饱和区后，延迟快速抬升

控制思路

监测 内存带宽 / PSI / CPI / LLC Miss

判定 识别带宽进入高位饱和区

动作 动态 MBA、热点重调度

QoS 保障 - 动态内存带宽调控



- 1 采集

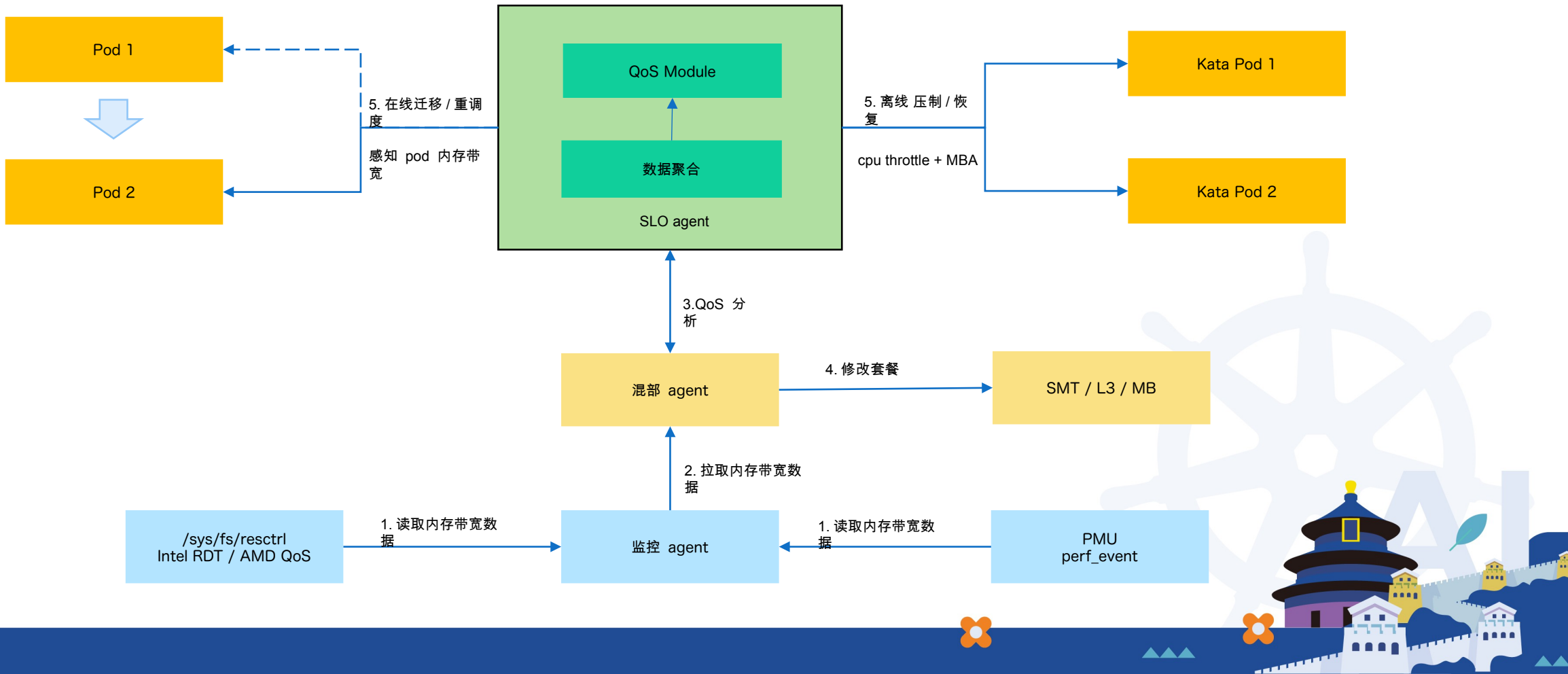
主机 / Pod 内存带宽数据
- 2 检测

监控 agent 汇总到 SLO agent
- 3 归因

SLO 异常 + 内存带宽阈值联动
- 4 处置

CPU throttle + MBA 压制 / 恢复
- 5 验证

SLO 是否恢复、结果是否符合预期



Part 04

未来规划

干扰检测、QoS 保障、AI Agent 技术



未来规划

- **干扰检测**：不再依赖 high level 应用 RT 指标和应用内部数据，构建更底层、更通用的干扰检测能力，指标精度由秒级提升至毫秒级。
- **QoS 保障**：在调度 / 重调度中引入资源画像与干扰风险特征，覆盖内存带宽、LLC、IO 等关键资源，实现动态、精细化的 QoS 调控。
- **AI Agent 技术**：在 K8s 混部场景中的落地，重点探索混部治理、异常诊断与自愈等方向。



参考

1. GOOGLE-WIDE PROFILING:A CONTINUOUS PROFILING INFRASTRUCTURE FOR DATA CENTERS <http://static.googleusercontent.com/media/research.google.com/zh-CN//pubs/archive/36575.pdf>
2. Heracles: Improving Resource Efficiency at Scale <https://static.googleusercontent.com/media/research.google.com/zh-CN//pubs/archive/43792.pdf>
3. 面向多应用混部的性能保障方法综述 <https://crad.ict.ac.cn/article/doi/10.7544/issn1000-1239.202220333>
4. Hardware-Level QoS Enforcement Features: Technologies, Use Cases, and Research Challenges <https://arxiv.org/html/2505.15542v1>
5. Brief Announcement: BALM: QoS-Aware Memory Bandwidth Partitioning for Multi-Socket Cloud Nodes https://dcatkth.github.io/papers/BALM_SPAA_2021_camera_ready.pdf



Thanks.

